CAutomation Project Engineering Contract

Authoritative platform-level engineering contract

Status: Final authoritative project-level platform contract for CAutomation. This document is intentionally platform-level. Module-specific requirements, screens, entities, endpoints, data fields, schemas, workflows, and acceptance details belong in module-level contracts. When a project-level rule and a module-level detail conflict, the project-level rule controls unless this project-level contract is formally changed.

Authoring rule: every section is written to be decision-complete for platform-level architecture. If required information is missing at module level, the AI must report the gap instead of inventing module behavior.

1. Document Purpose and Engineering Boundary

This contract defines how the CAutomation platform must be engineered at project level. It gives the AI shared engineering authority for architecture, stack, security, API, data, logging, validation, testing, AI generation, deployment, and reporting.

- Use this document for engineering decisions inherited by every module.
- Do not use it to define module-specific endpoints, schemas, fields, screens, workflows, reports, or domain algorithms.
- If module technical detail is missing, the AI must report a module-contract gap rather than placing that detail in the project contract.
- Engineering decisions here are mandatory unless this project-level contract is formally changed.

2. Engineering Principles

These principles are mandatory across the generated application.

- Contract-first: approved project-level and module-level contracts are the source of truth.
- Validation-first: invalid, unsupported, ambiguous, incomplete, or unapproved inputs fail before downstream generation or acceptance.
- Traceability-first: generated artifacts, execution reports, logs, validation results, approvals, and exports reference their source contracts and execution identity.
- Separation of concerns: UI, API, application services, domain model, repositories, persistence, AI execution, validation, reporting, and export responsibilities remain distinct.
- Modular but integrated: modules are independently understandable but operate inside one shared platform shell.
- Explicit over implicit: hidden defaults, inferred permissions, unrecorded approvals, and silent fallback behavior are not acceptable.
- Whole-file generation: AI-generated source changes are complete files with manifest entries unless a later project-level rule permits another mode.

3. Platform Architecture

CAutomation must be generated as an end-to-end web application with explicit layers. Layer boundaries are architectural rules, not naming suggestions.

Layer	Mandatory responsibility	Must not contain
Frontend web application	React TypeScript UI for navigation, administration, contract management, execution review, reports, approvals, and exports.	Database access or authoritative business-rule enforcement.
Backend API	FastAPI REST API exposing authenticated platform and module operations.	Frontend rendering logic or direct AI provider coupling in controllers.
Application services	Use-case orchestration, authorization checks, lifecycle transitions, validation coordination, and transaction boundaries.	Raw SQL scattered through controllers or UI concerns.
Domain model	Shared concepts: Client, Project, Module, Contract, Pipeline, Task, Execution, Report, Deliverable, Role, Permission, Approval.	Framework-specific persistence or transport details.
Repositories	Persistence access and query encapsulation.	Business orchestration or authorization policy decisions.
Database	PostgreSQL persistence for platform state, lifecycle, ownership, traceability, reports, and deliverables.	Business logic implemented only as database side effects.
AI execution layer	Provider-independent execution, staging, prompt, raw response, manifest, and generated artifact handling.	Direct writes to accepted project state without validation/apply control.
Validation/reporting	Structured checks and machine-readable reports for inputs, outputs, artifacts, executions, and acceptance blockers.	Unstructured messages as the only evidence.
Export layer	Packaging approved deliverables with reports, manifests, and provenance.	Bypassing approval or validation state.

4. Technology Stack

The stack must be explicit so generated code remains consistent across modules.

Area	Decision	Rationale / boundary
Frontend	React with TypeScript.	Single web UI with typed components and typed API clients.
Backend	Python FastAPI.	Clear REST APIs, validation models, and service layering.
Database	PostgreSQL.	Relational persistence for platform state, traceability, lifecycle, and audit needs.
Data validation	Pydantic-style request, response, and internal validation models.	Do not rely on untyped dictionaries for core API contracts.
Testing	Unit, functional, validation, task, pipeline, AI Engine, and CAutomation-level runners where applicable.	Runners must produce structured reports for GUI consumption.
Documents	PDF as primary supported source contract format at this stage.	DOCX support may exist but must not drive equal current engineering effort.
AI provider	Provider abstraction with provider-specific integrations behind a stable contract.	Application logic must not depend directly on one provider implementation.

5. Project Structure Standards

Generated artifacts must preserve predictable locations so platform code, tests, reports, and future GUI tooling can find assets consistently.

- The root project name is CAutomation. Legacy names must not be generated.
- Project-level contracts belong under project-level contract locations; module-level contracts belong under each module.
- Each pipeline and task keeps executable code, configuration, tests, fixtures, reports, and documentation discoverable by execution level.
- Generated output separates source contracts, normalized inputs, staging workspaces, reports, manifests, logs, exports, and accepted deliverables.
- No module may write directly into another module ownership area except through approved platform APIs or generated integration contracts.
- Runtime reports and generated artifacts must not be confused with authoritative source contracts.

6. Backend Standards

Backend generation must follow layered service architecture and explicit validation.

- API routes accept and return typed request/response models.
- Routes delegate business behavior to services; they do not implement complex domain workflows directly.
- Services enforce authorization, lifecycle validity, cross-module rules, validation coordination, and transaction boundaries.
- Repositories encapsulate persistence access and return domain or data-transfer objects, not UI-specific structures.
- Validation failures return structured errors with actionable messages and machine-readable categories/codes where practical.
- Long-running or pipeline-like operations expose execution state rather than blocking silently.
- File and artifact handling preserves provenance, safe paths, ownership context, and controlled lifecycle state.

7. Frontend Standards

Frontend generation must produce a coherent platform UI rather than isolated module screens.

- The UI provides a shared application shell with navigation for clients, projects, modules, contracts, executions, reports, deliverables, approvals, waivers, and exports.
- Screens show current context: selected client, selected project, selected module where applicable, status, owner, blockers, provenance, and next action.
- Frontend code uses typed API clients or typed request/response shapes; ad hoc JSON assumptions are not acceptable for platform concepts.
- Validation and execution errors are visible to users in business language, with enough technical detail to act when appropriate.
- Reusable components are used for tables, status badges, evidence panels, approval actions, report viewers, artifact lists, and provenance summaries.
- The frontend must not implement security by hiding UI only; backend authorization is authoritative.

8. Database Standards

Database design must support isolation, lifecycle state, traceability, and auditability across all modules.

- Core records include stable identity, ownership context, lifecycle/status fields, timestamps, and actor references where applicable.
- Client-owned and project-owned data carry enough keys to enforce visibility and authorization.
- Contract, normalized input, execution, report, deliverable, approval, waiver, and export records preserve source/version relationships.
- Use migrations for schema changes; generated code must not assume manual database edits.
- Prefer soft deletion, archive, or supersession for business-significant records. Hard deletion must be explicitly justified.
- Database constraints support integrity, but business workflows must still be expressed in services.

9. API Standards

The API must be consistent enough for frontend and backend generation without inventing patterns.

- REST resources reflect platform concepts: clients, projects, modules, contracts, pipelines, tasks, executions, reports, deliverables, approvals, waivers, exports, users, roles, permissions.
- Endpoints use consistent plural nouns, stable identifiers, and explicit client/project/module context where required.
- Requests and responses are typed and documented through the backend framework where possible.
- Errors are structured and distinguish validation failure, authorization failure, authentication failure, missing resource, conflict/lifecycle violation, unsafe path, and unexpected server failure.
- Operations that change approval, execution, export, waiver, or lifecycle state are explicit commands rather than hidden side effects of read operations.
- Pagination, filtering, and sorting are available for list views that can grow over time.
- APIs must not expose cross-client data through identifiers, filters, reports, or artifact paths.

10. Security and Authorization Principles

Security is platform-level and cannot be weakened by module contracts.

- Authentication is required for all non-public platform operations.
- Authorization is enforced on the backend using roles, permissions, ownership context, and lifecycle state.
- Client isolation is mandatory. Users must not read or mutate other clients data without explicit cross-client platform authority.
- Project membership limits project visibility and actions.
- Approval, waiver, export, administrative, and execution operations require specific permissions and must be auditable.
- Secrets, provider keys, and deployment configuration must not be committed into source contracts or generated source files.
- File paths, uploaded documents, normalized documents, generated artifacts, reports, and exports are validated to prevent unsafe access outside approved workspace boundaries.

11. Quality and Test Standards

Quality standards must make generated output inspectable and suitable for future GUI reporting.

- Unit tests verify services, validators, repositories, API models, and components.
- Functional tests verify platform workflows across APIs and important frontend/backend seams.
- Validation tests verify contract normalization, input readiness, artifact manifests, reports, blockers, and acceptance rules.
- Task, pipeline, AI Engine, and CAutomation-level runners use names that identify execution level clearly: run_task_tests, run_pipeline_tests, run_ai_engine_tests, and run_cautomation_tests.
- Test reports are structured and machine-readable enough for later web GUI consumption.
- A passing implementation is not enough if reports, manifests, traceability, validation evidence, or acceptance blockers are missing.
- Regression tests protect platform concepts and cross-module rules from accidental drift.

12. Logging, Monitoring, and Reports

The platform must make operational and generation behavior understandable without relying on hidden console output.

- Use structured logging for important platform, security, validation, execution, provider, export, and lifecycle events.
- Logs include execution identifiers, project context, module context, task identity, actor, and error category when applicable.

- Reports are generated for normalization, validation, execution, apply/export, and final run summaries where relevant.
- Reports distinguish success, warning, failure, skipped, blocked, and waived states.
- Business users inspect reports through the platform UI; engineers can inspect raw structured report files.
- Do not use print statements as the primary operational reporting mechanism.
- Reports must not claim success when required evidence is missing.

13. AI Generation Standards

AI-generated behavior must remain bounded by approved contracts and controlled workspaces.

- The AI uses approved contracts and normalized inputs as authority; it must not read unrelated live repository files to invent requirements.
- Generation writes to a staging workspace first, preserving prompt, raw response, manifest, generated files, and generation report.
- Generated artifacts are whole files with explicit paths and manifest entries.
- Schema mismatch, missing required fields, unsafe paths, unsupported operations, or out-of-scope changes block apply/acceptance.
- Provider-specific details remain behind a provider abstraction.
- If a required decision is missing from contracts, the generation report identifies the gap instead of silently inventing the answer.
- AI outputs preserve CAutomation naming and must not reintroduce legacy project names.

14. Cross-Module Engineering Rules

Modules must integrate through shared platform rules rather than duplicating platform infrastructure.

- Modules inherit authentication, authorization, client/project context, contract lifecycle, execution traceability, validation reporting, approval, waiver, export, and logging rules.
- Modules may define their own domain entities and workflows only inside module contracts.
- A module must not define a parallel user model, permission model, approval model, execution model, report model, contract model, or validation model.
- Shared UI components and backend services are reused for common platform concepts.
- Cross-module references are explicit and validated; hidden coupling through file paths, duplicated identifiers, or unapproved shared state is not acceptable.
- Module-specific behavior may extend platform behavior only where it does not weaken project-level rules.

15. Deployment and Environment Principles

Deployment expectations must be explicit enough to avoid environment-specific invention.

- The application supports separate configuration for local development, test, and production-like environments.
- Secrets and provider credentials come from environment/configuration mechanisms, not hard-coded files.
- Database migrations and initial setup are repeatable.
- Generated reports, staging workspaces, logs, normalized inputs, and exports use configurable safe locations.
- The platform should be container-friendly, but exact production infrastructure choices are outside this contract unless defined later.
- Deployment preserves the ability to inspect logs, reports, generated artifacts, manifests, and validation evidence.

16. Engineering Acceptance Criteria

The generated platform is technically acceptable only when platform-level engineering rules are implemented consistently across modules.

- Frontend, backend, database, validation, reporting, AI execution, security, and export behavior work together as one CAutomation platform.
- Project-level contracts and module-level contracts remain synchronized in generated outputs without moving module details into project-level locations.
- Security, authorization, client isolation, project membership, safe paths, and lifecycle permissions are enforced server-side.
- Every business-significant generated artifact is traceable to approved source contracts, normalized inputs, execution identity, manifests, and validation evidence.
- Failed validation blocks acceptance and produces actionable structured reports.

- Tests and runners produce structured reports suitable for later GUI consumption.
- No generated file uses legacy project names or module-specific behavior in project-level locations.

17. Engineering Glossary

The AI must use these engineering terms consistently.

Term	Engineering definition
Application service	Backend layer that orchestrates business use cases, authorization, validation, transactions, and lifecycle transitions.
Repository	Backend layer that encapsulates persistence access and queries.
Execution identity	Stable identifier connecting a pipeline/task run to inputs, outputs, logs, reports, manifests, and artifacts.
Manifest	Structured inventory of generated artifacts, paths, checksums, provenance, and metadata.
Staging workspace	Controlled location where AI-generated output is written before validation/apply/acceptance.
Validation report	Machine-readable evidence describing checks, results, failures, warnings, blockers, and waivers.
Apply	Controlled promotion of validated generated artifacts into an accepted target location.
Structured error	Error response with category, message, details, and where practical a machine-readable code.
Provider abstraction	Stable interface hiding provider-specific AI implementation details.
Scope drift	Any generated change outside the approved iteration boundary.
Safe path	A validated file path constrained to approved workspace boundaries and ownership context.